

bok25

基
礎
班

、**考研数据結構**

选择排序

简单选择排序



选择排序

简单选择排序

基本思想

每一趟排序在无序序列中找到最小的元素与无序序列的第一个元素交换
直到整体有序



选择排序

简单选择排序

基本思想

每一趟排序在无序序列中找到最小的元素与无序序列的第一个元素交换直到整体有序

```
void SelectSort(SqList &L)
{ //对顺序表L进行简单选择排序
  for(i=1; i<L.length; ++i)
  { //在L[i..L.length] 中选择关键字最小的记录
    k=i;
    for(j=i+1; j<=L.length; ++j){
      if(L[j]<L[k]) k=j; } //k指向此趟排序中关键字最小的记录
    if(k!=i){
      t=L[i];
      L[i]=L[k];
      L[k]=t; } //交换r[i]与r[k]
  } //for
}
```

49	38	65	97	49	13	27	76
----	----	----	----	----	----	----	----



选择排序

简单选择排序

基本思想

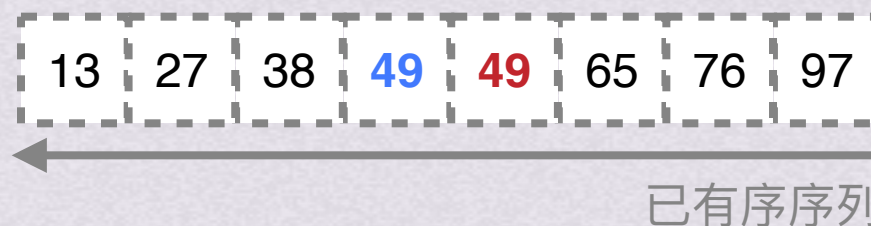
每一趟排序在无序序列中找到最小的元素与无序序列的第一个元素交换直到整体有序

```
void SelectSort(SqList &L)
{ //对顺序表L进行简单选择排序
  for(i=1; i<L.length; ++i)
  { //在L[i..L.length] 中选择关键字最小的记录
    k=i;
    for(j=i+1; j<=L.length; ++j){
      if(L[j]<L[k]) k=j; } //k指向此趟排序中关键字最小的记录
    if(k!=i){
      t=L[i];
      L[i]=L[k];
      L[k]=t; } //交换r[i]与r[k]
  } //for
}
```

空间复杂度: $O(1)$

时间复杂度不论好坏情况始终是 $O(n^2)$

不稳定!





选择排序

堆排序



选择排序

堆排序

大根堆: $L(i) \geq L(2i)$ 且 $L(i) \geq L(2i+1)$

小根堆: $L(i) \leq L(2i)$ 且 $L(i) \leq L(2i+1)$

可以将大根堆视为一棵每个根节点均大于等于其孩子节点的完全二叉树

可以将小根堆视为一棵每个根节点均小于等于其孩子节点的完全二叉树

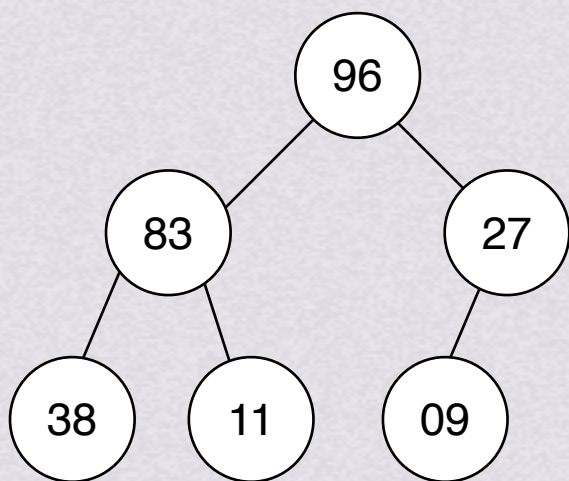


选择排序

堆排序

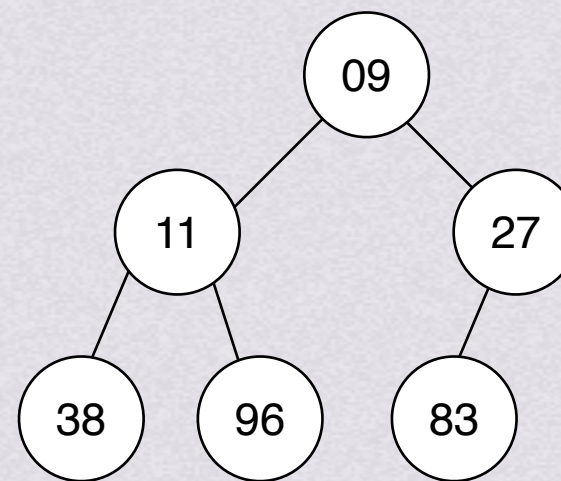
可以将大根堆视为一棵每个根节点均大于等于其孩子节点的完全二叉树

可以将小根堆视为一棵每个根节点均小于等于其孩子节点的完全二叉树



96	83	27	38	11	09
----	----	----	----	----	----

大根堆



09	11	27	38	96	83
----	----	----	----	----	----

小根堆



选择排序

堆排序

可以将大根堆视为一棵每个根节点均大于等于其孩子节点的完全二叉树

可以将小根堆视为一棵每个根节点均小于等于其孩子节点的完全二叉树

堆排序：每次输出堆顶元素，然后将堆底元素送入堆顶
调整堆来继续保持大根堆性质，再输出堆顶元素，直到整体有序



选择排序

堆排序

堆排序：每次输出堆顶元素，然后将堆底元素送入堆顶，调整堆来继续保持大根堆性质，再输出堆顶元素，直到整体有序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素

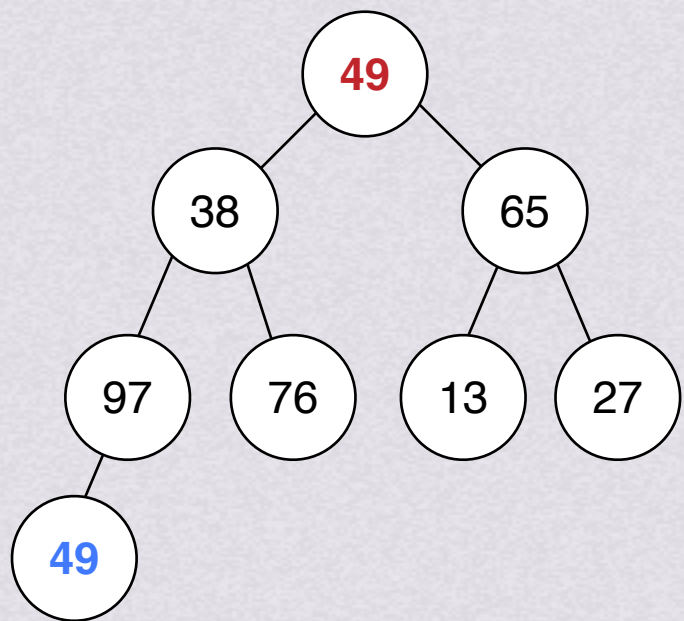


选择排序堆排序

堆排序：每次输出堆顶元素，然后将堆底元素送入堆顶，调整堆来继续保持大根堆性质，再输出堆顶元素，直到整体有序

分为两个步骤：**1.建堆**

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始

根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

无序序列

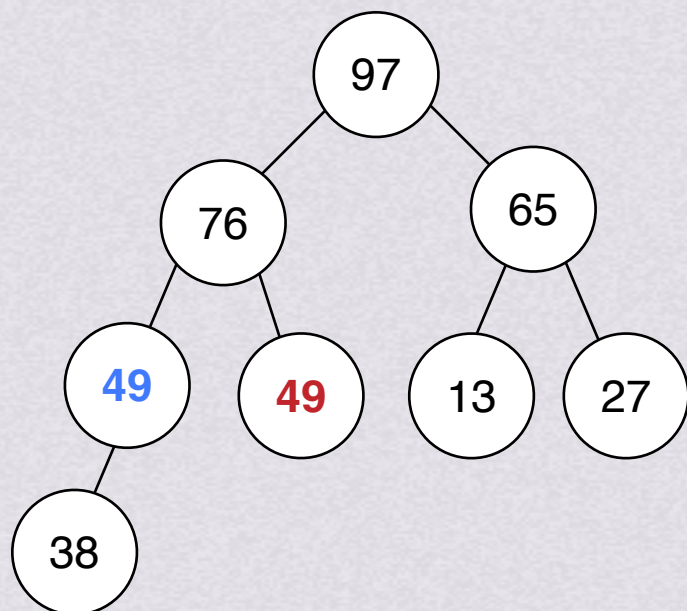
49	38	65	97	76	13	27	49
----	----	----	----	----	----	----	----



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换
2.逐个从后往前让所有子树满足大根堆定义

建堆完成！
开始交换元素！

无序序列

数组下标

-	97	76	65	49	49	13	27	38
0	1	2	3	4	5	6	7	8

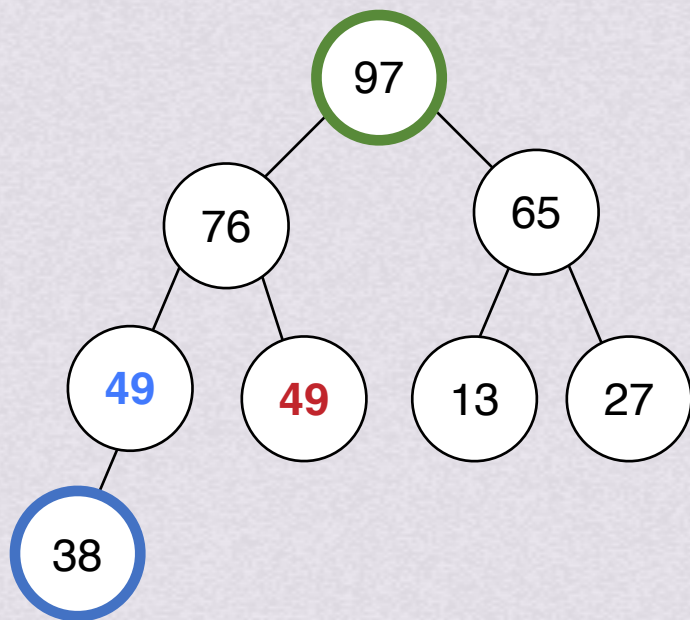
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：

1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标

-	97	76	65	49	49	13	27	38
0	1	2	3	4	5	6	7	8

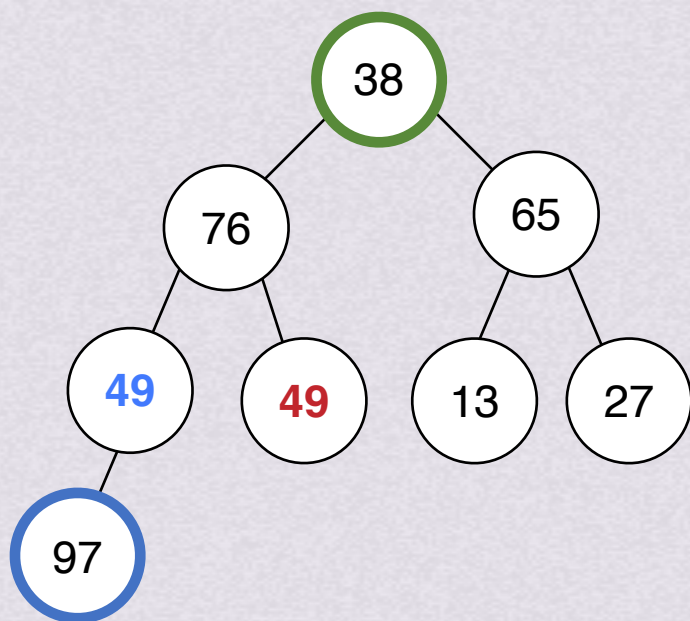
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标

-	38	76	65	49	49	13	27	97
0	1	2	3	4	5	6	7	8

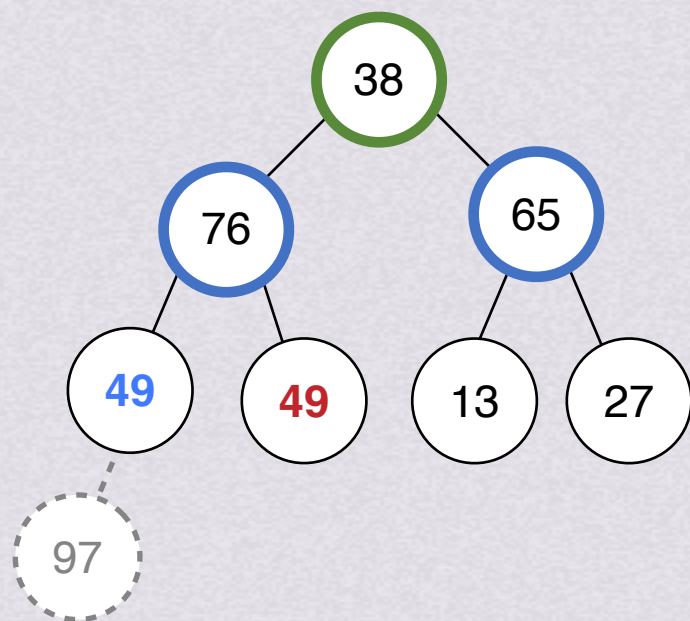
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

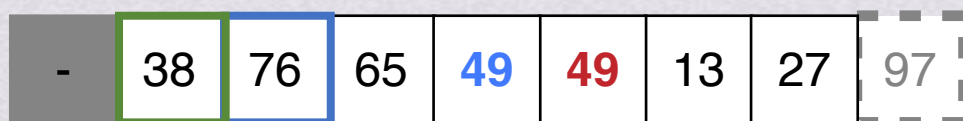
2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



0 1 2 3 4 5 6 7 8

待排序元素 已有序元素

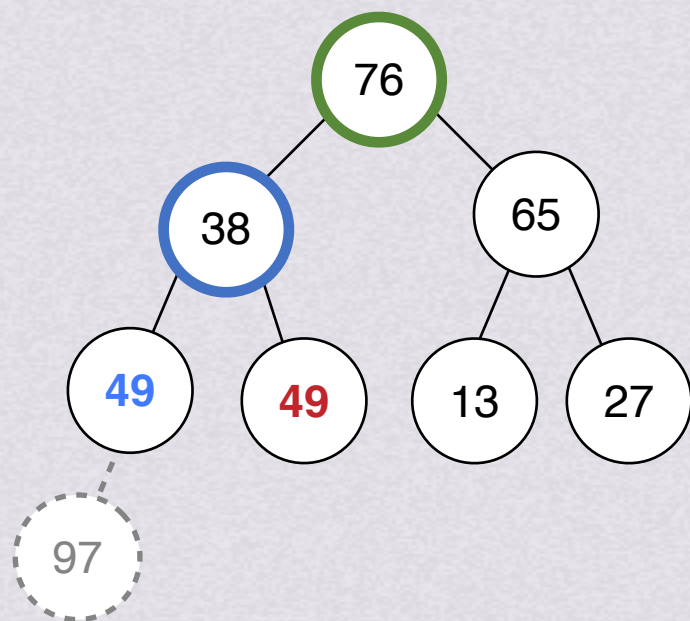
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

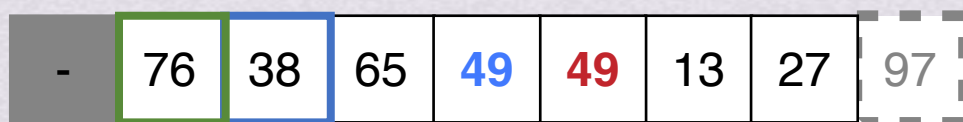
2.找到堆底元素

3.交换

4.把剩余的*i*-1个元素调整成堆

无序序列

数组下标



待排序元素 已有序元素

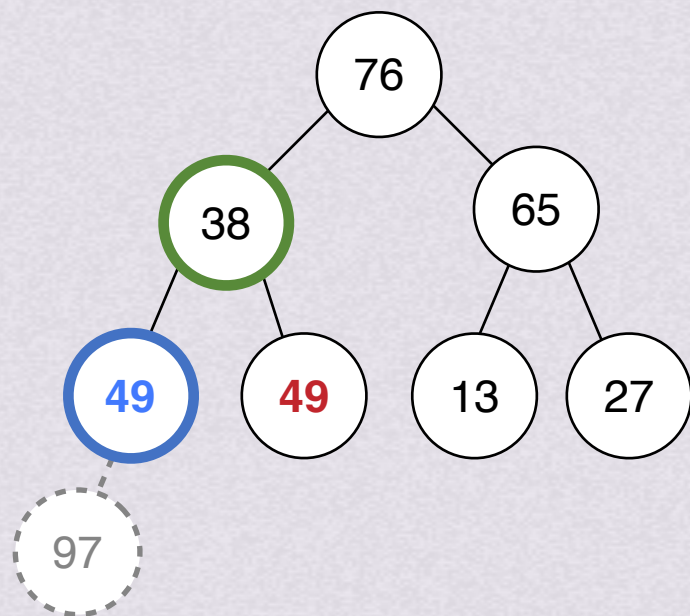
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

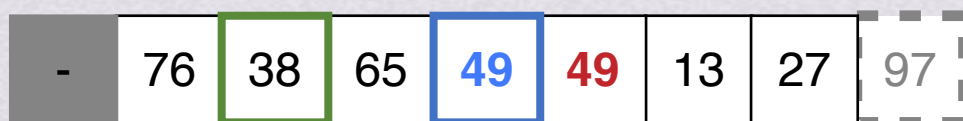
2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

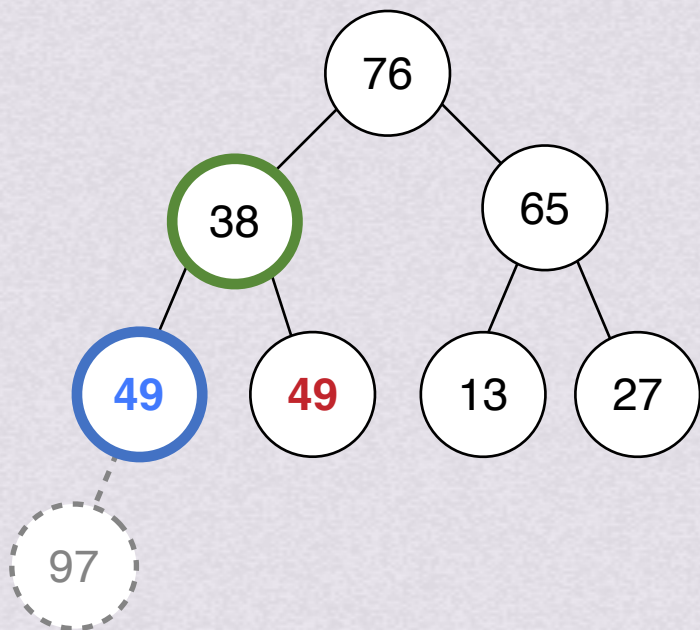
无序序列

数组下标



待排序元素 已有序元素

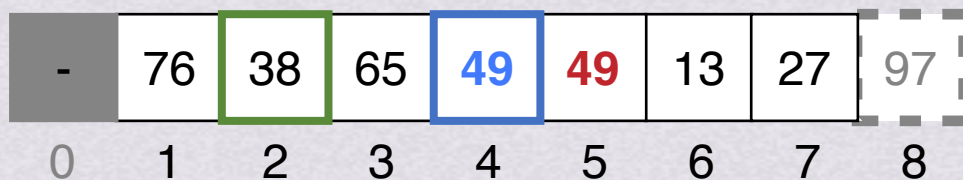
*0位置用于暂存根节点以完成交换操作，此处不更新



```
void HeapAdjust(SqList &L, int s, int m)
{ //假设L[s+1..m]已经是堆, 将L[s..m]调整为以L[s]为根的大根堆
  rc=L[s];
  for(j=2*s; j<=m; j*=2) //沿key较大的孩子节点向下筛选
  {
    //所以两孩子相同的时候选择左孩子
    if(j<m && L[j]<L[j+1]) ++j; //j为key较大的记录的下标
    if(rc>=L[j]) break; //rc应插入在位置s上
    L[s]=L[j];
    s=j;
  } //for
  L[s]=rc; //插入
}
```

无序序列

数组下标



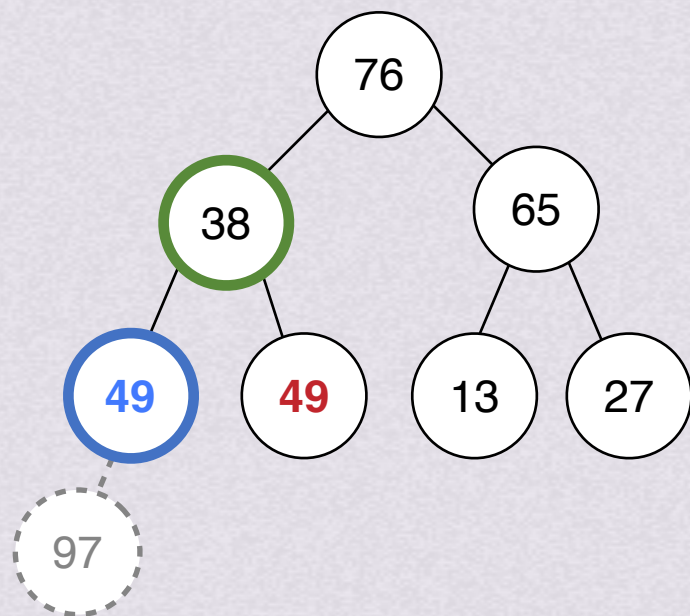
待排序元素 已有序元素



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

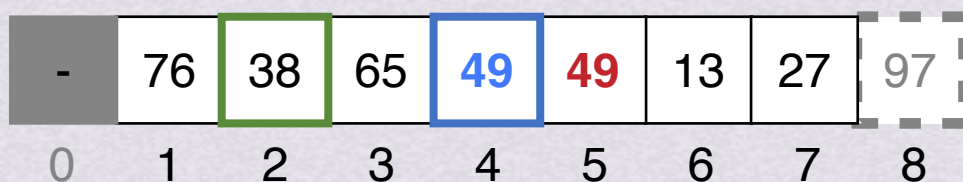
2.找到堆底元素

3.交换

4.把剩余的*i*-1个元素调整成堆

无序序列

数组下标



待排序元素 已有序元素

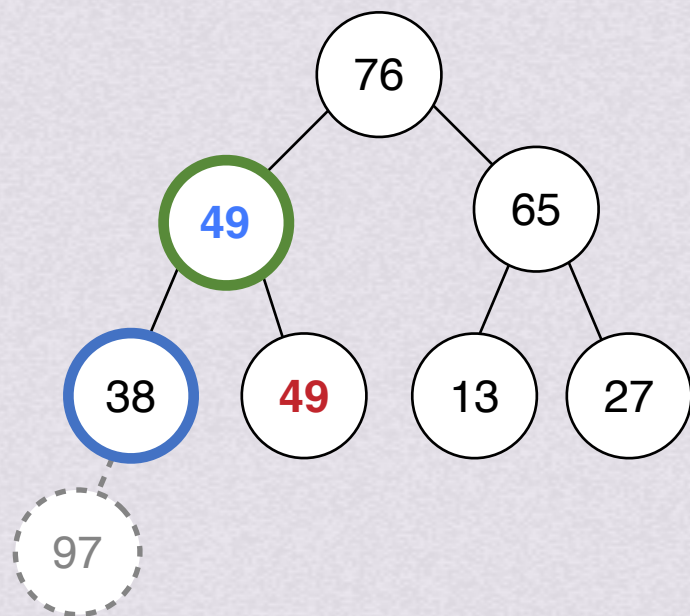
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

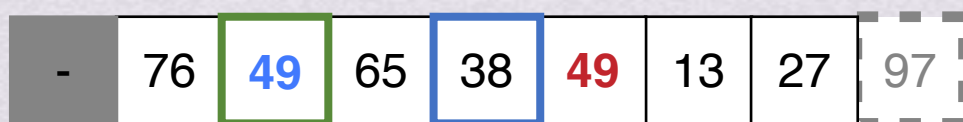
3.交换

4.把剩余的*i*-1个元素调整成堆

调整完毕，继续交换堆顶堆底元素

无序序列

数组下标



待排序元素 已有序元素

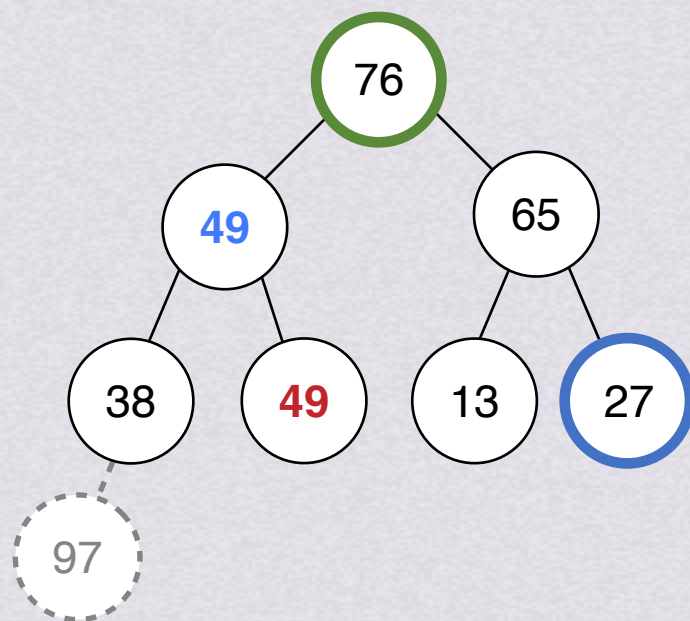
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：

1.找到堆顶元素

2.找到堆底元素

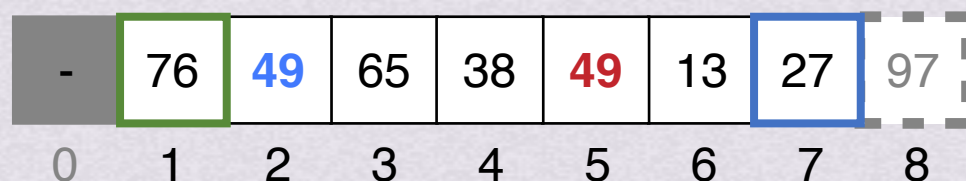
3.交换

4.把剩余的i-1个元素调整成堆

调整完毕，继续交换堆顶堆底元素

无序序列

数组下标



待排序元素

已有序元素

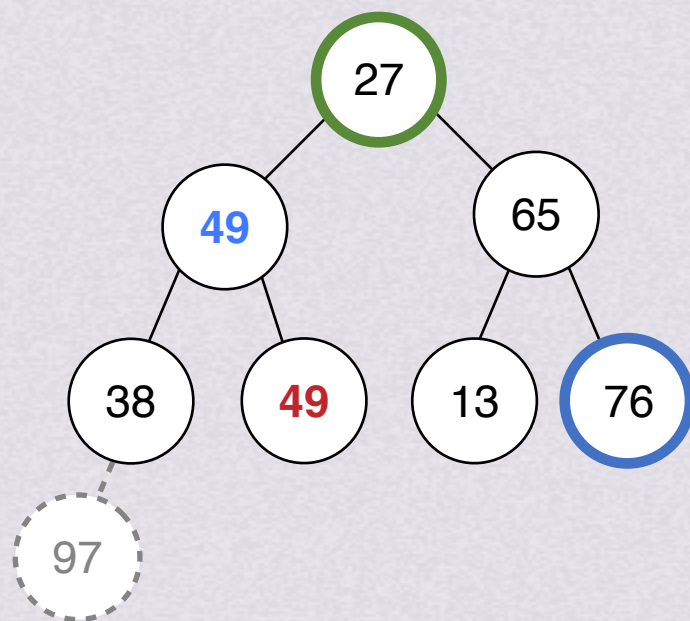
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

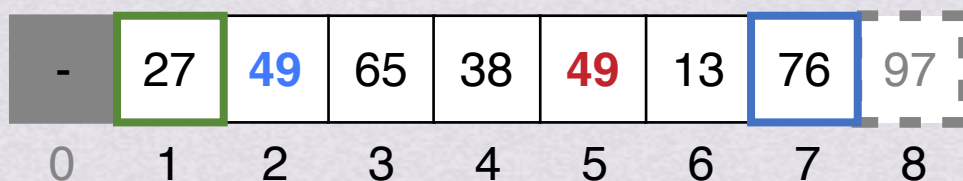
2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



待排序元素 已有序元素

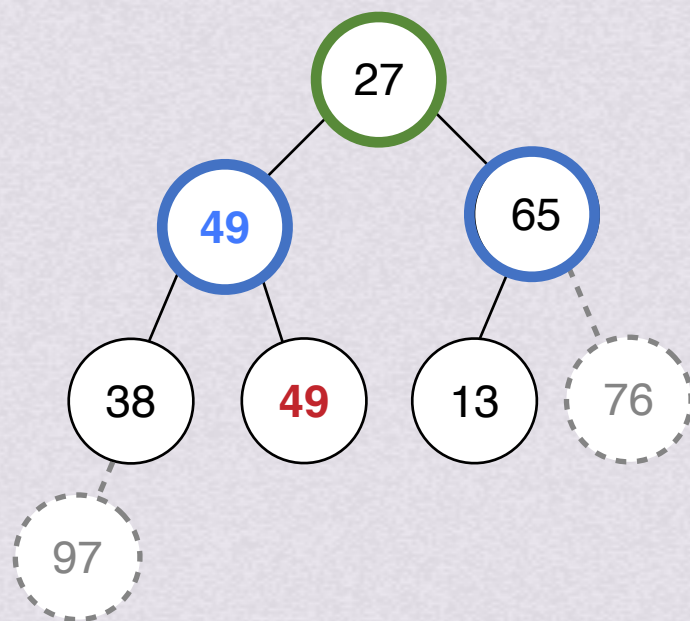
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的*i*-1个元素调整成堆

无序序列

数组下标



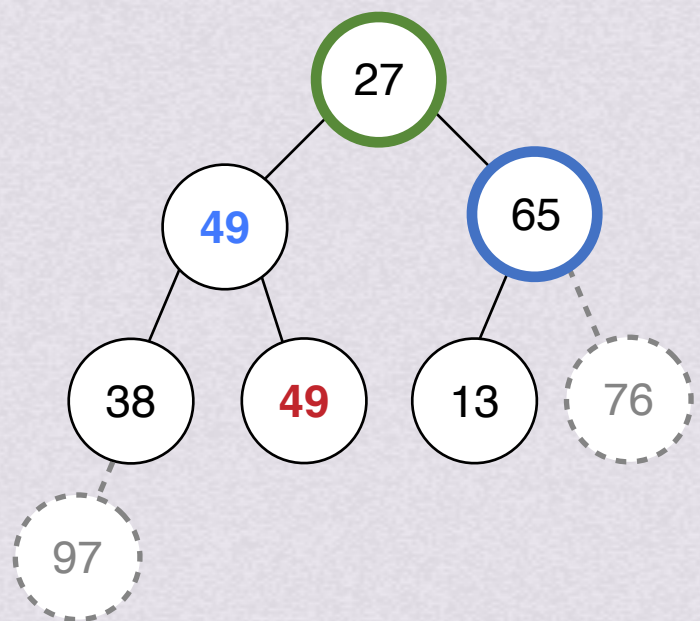
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



0 1 2 3 4 5 6 7 8

待排序元素 已有序元素

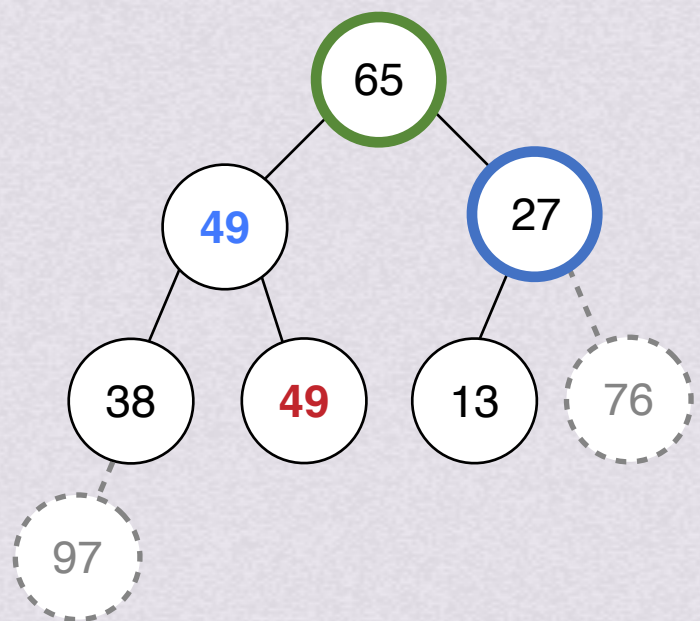
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的*i*-1个元素调整成堆

无序序列

数组下标



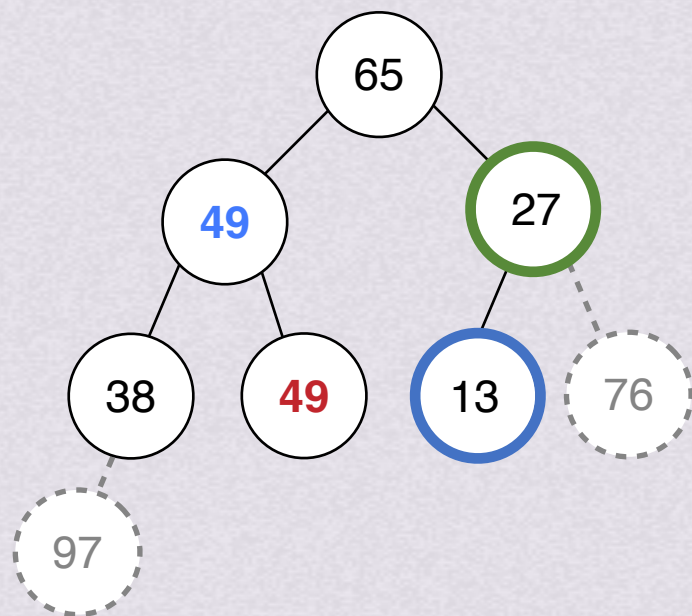
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

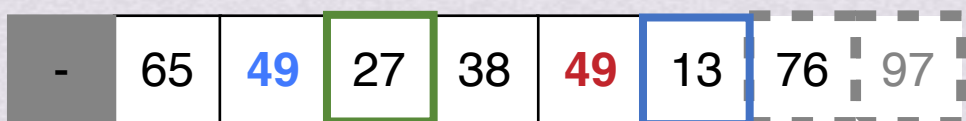
3.交换

4.把剩余的*i*-1个元素调整成堆

调整完毕，继续交换堆顶堆底元素

无序序列

数组下标



0 1 2 3 4 5 6 7 8

待排序元素 已有序元素

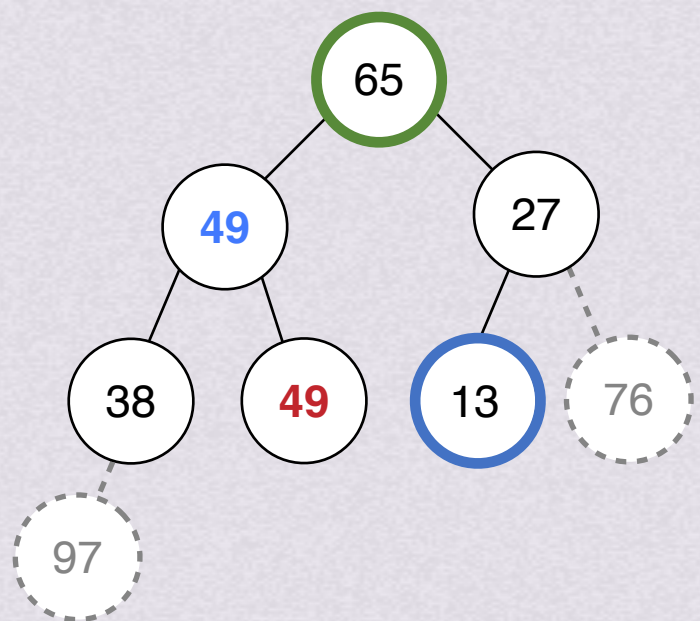
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：

1.找到堆顶元素

2.找到堆底元素

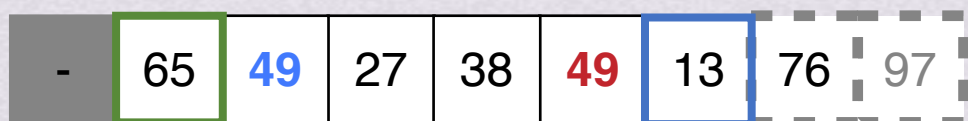
3.交换

4.把剩余的i-1个元素调整成堆

调整完毕，继续交换堆顶堆底元素

无序序列

数组下标



0 1 2 3 4 5 6 7 8

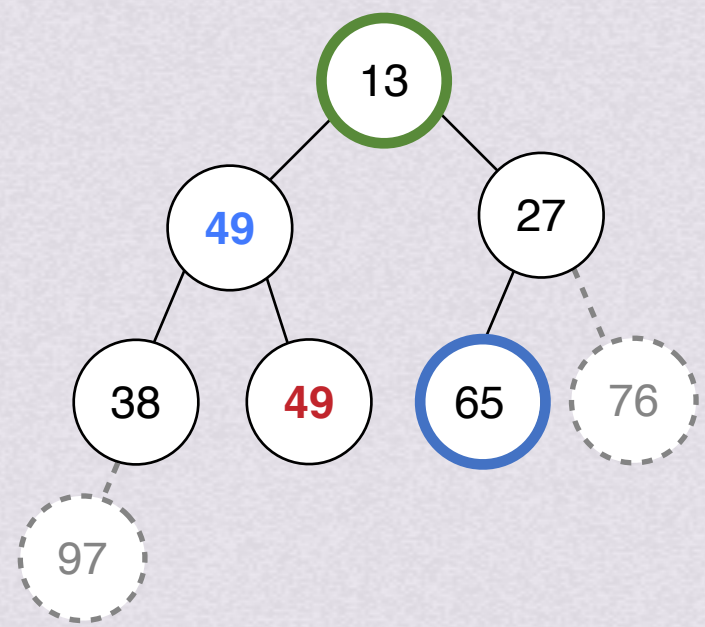
待排序元素 已有序元素

*0位置用于暂存根节点以完成交换操作，此处不更新

选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换
2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素
2.找到堆底元素
3.交换
4.把剩余的i-1个元素调整成堆

调整完毕，继续交换堆顶堆底元素



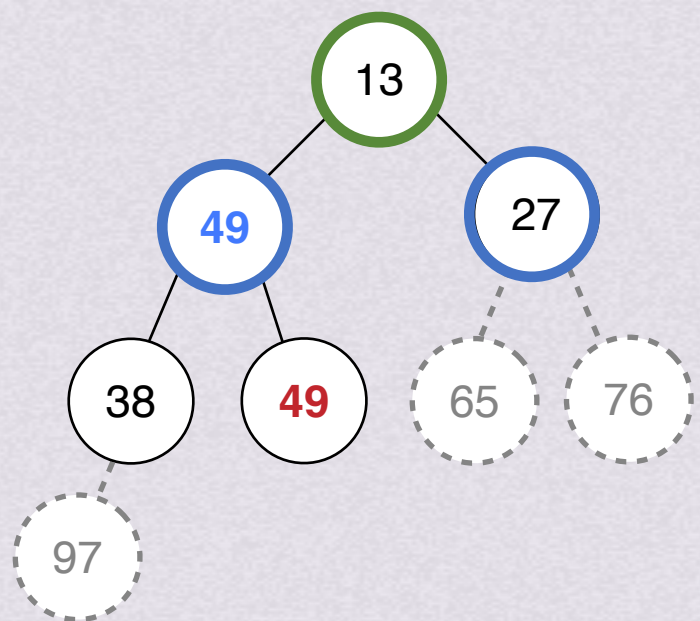
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的*i*-1个元素调整成堆

调整完毕，继续交换堆顶堆底元素

无序序列

数组下标

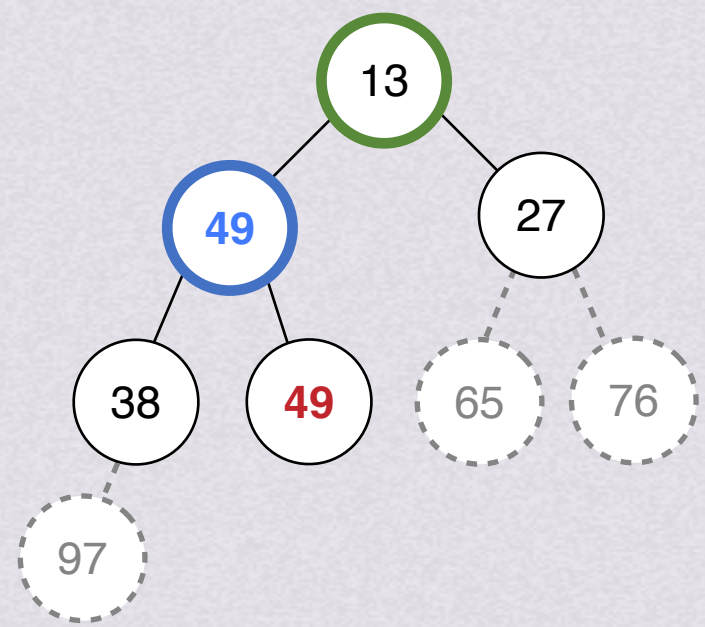


*0位置用于暂存根节点以完成交换操作，此处不更新

选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换
2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素
2.找到堆底元素
3.交换
4.把剩余的i-1个元素调整成堆



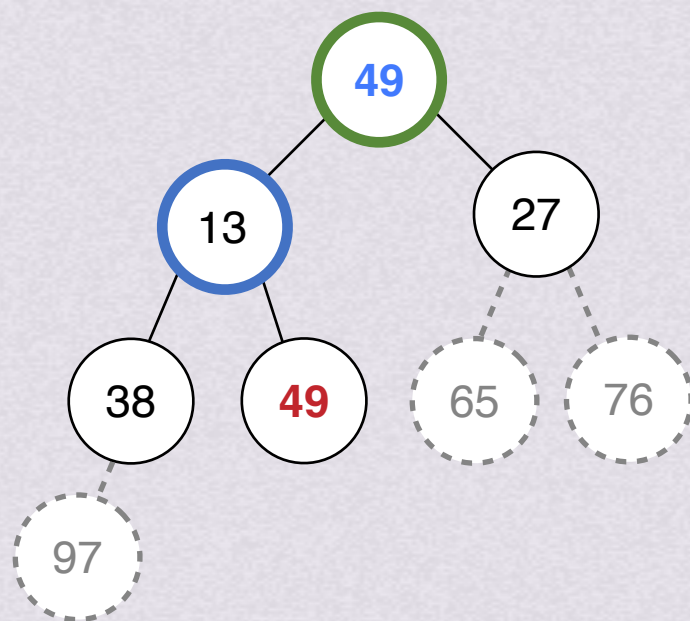
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



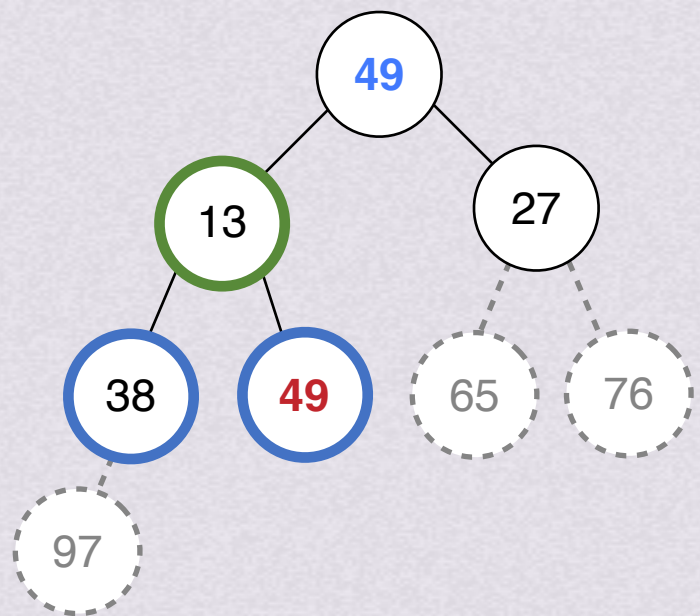
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



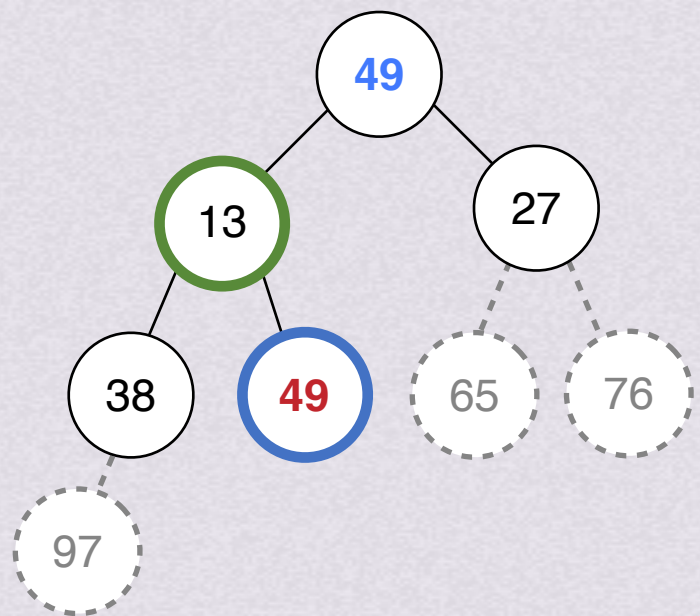
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的*i*-1个元素调整成堆

无序序列

数组下标



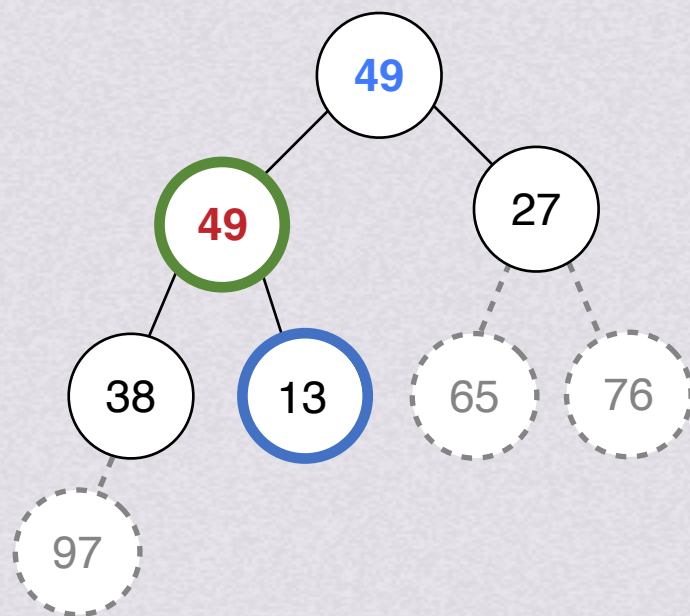
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

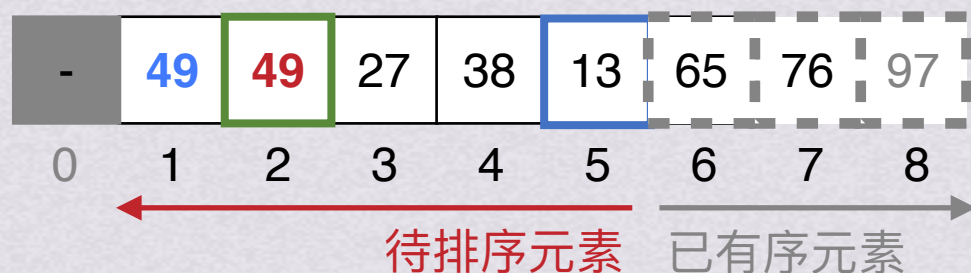
2.找到堆底元素

3.交换

4.把剩余的*i*-1个元素调整成堆

无序序列

数组下标



*0位置用于暂存根节点以完成交换操作，此处不更新

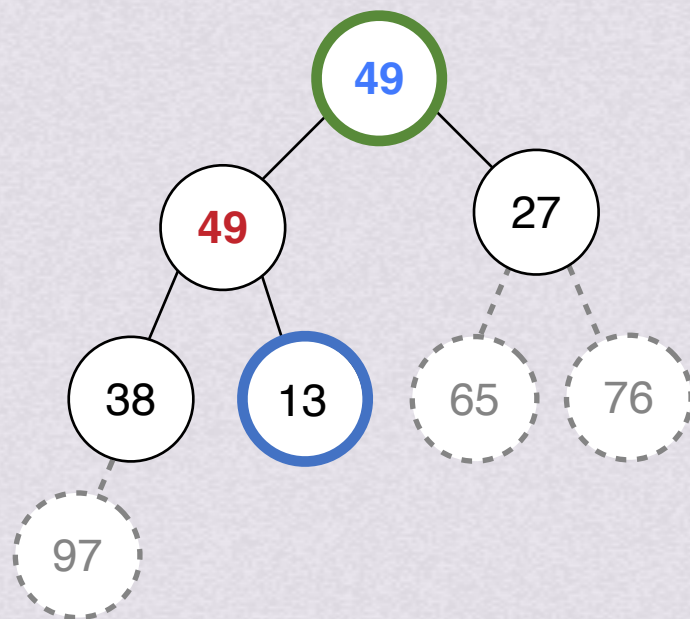


选择排序

堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：

1.找到堆顶元素

2.找到堆底元素

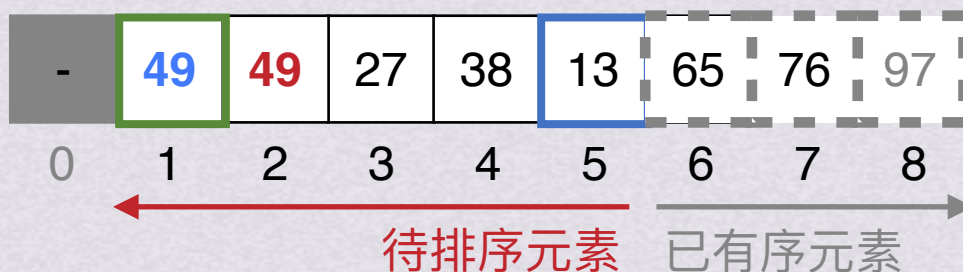
3.交换

4.把剩余的i-1个元素调整成堆

调整完毕，继续交换堆顶堆底元素

无序序列

数组下标

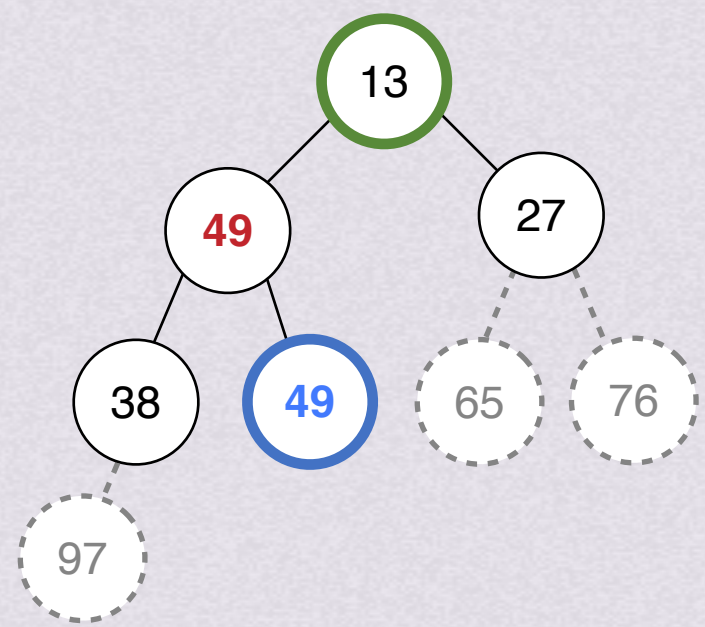


*0位置用于暂存根节点以完成交换操作，此处不更新

选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换
2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素
2.找到堆底元素
3.交换
4.把剩余的i-1个元素调整成堆

调整完毕，继续交换堆顶堆底元素



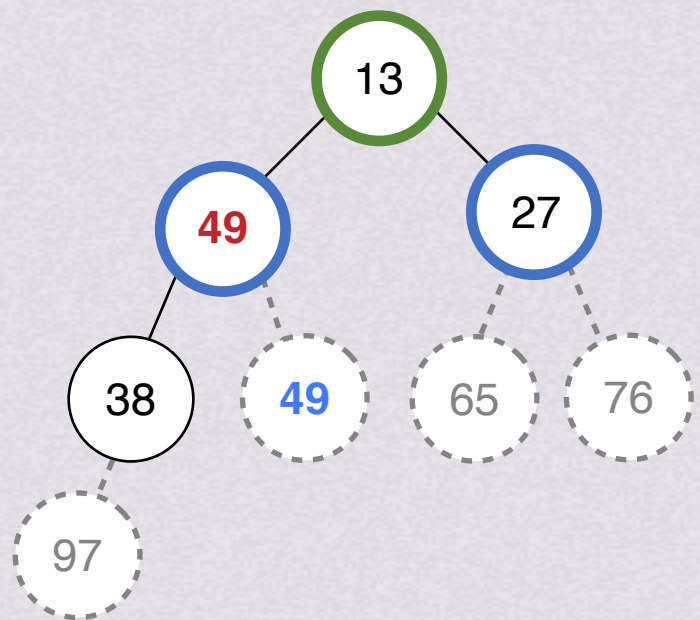
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

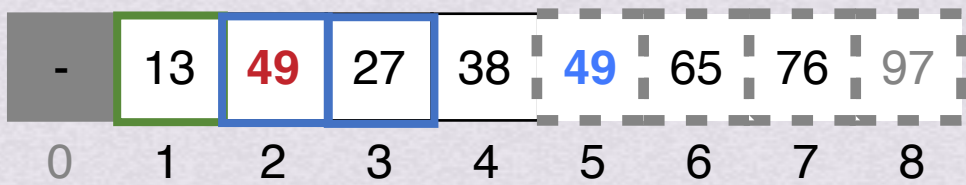
2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



待排序元素 已有序元素

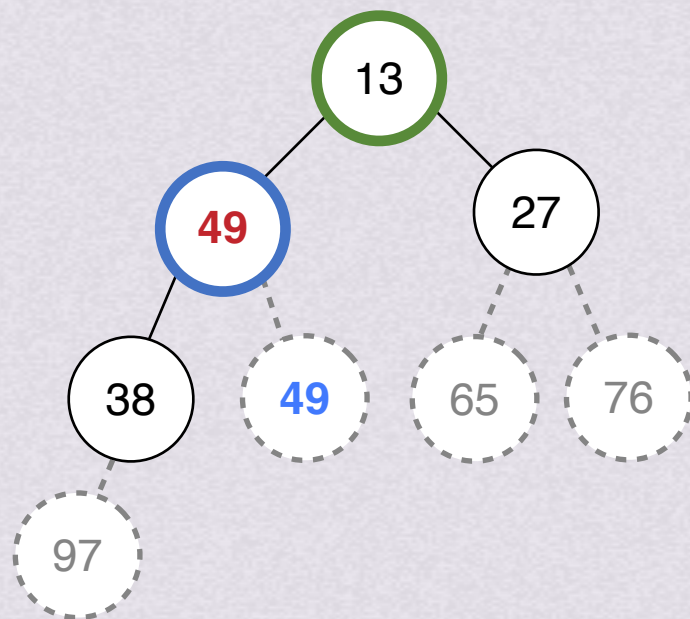
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

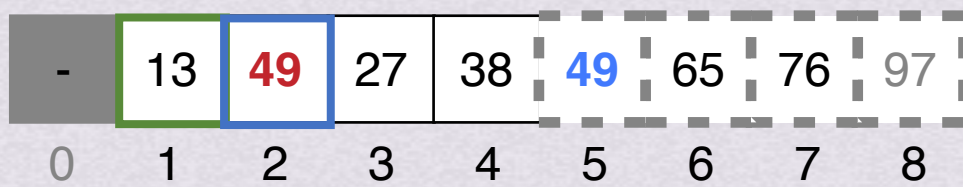
2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



待排序元素 已有序元素

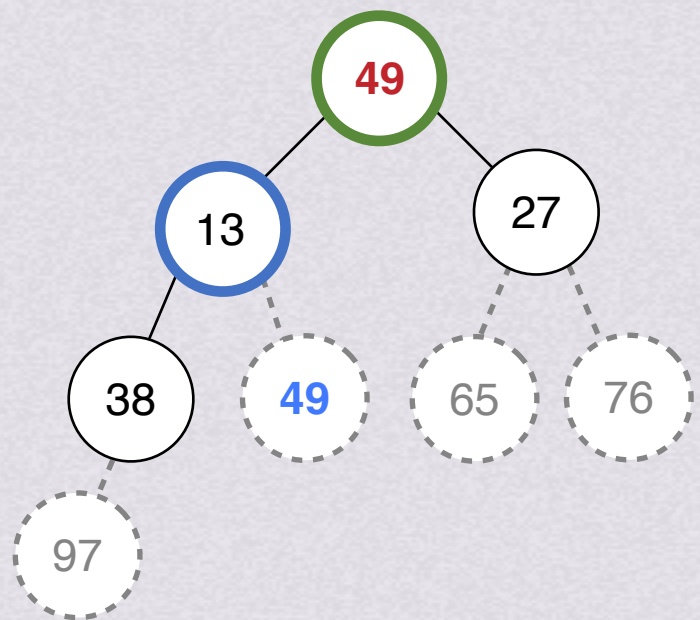
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

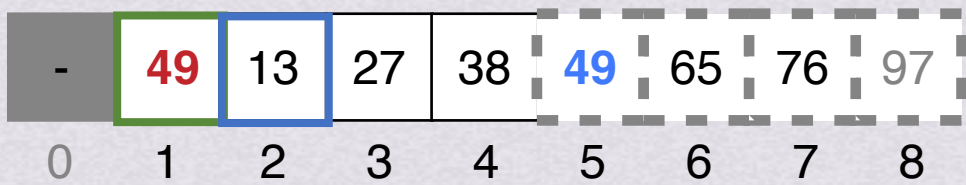
2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



待排序元素 已有序元素

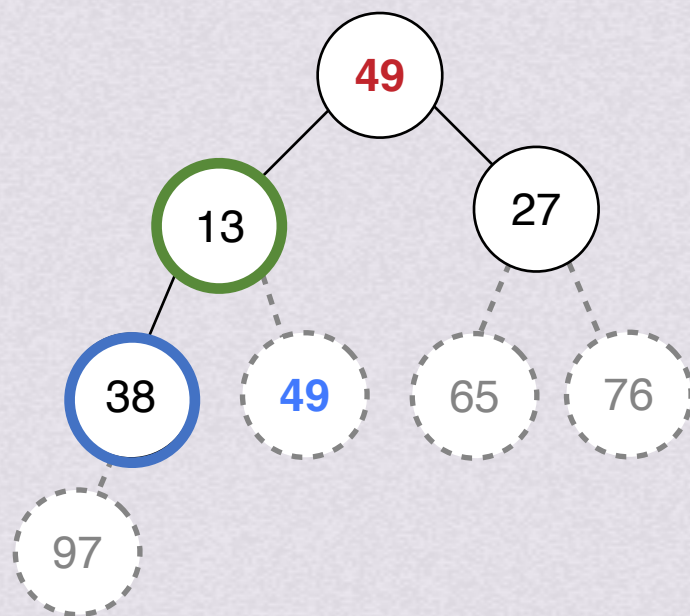
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

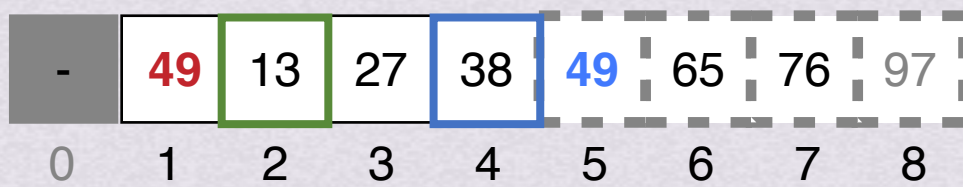
2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



待排序元素 已有序元素

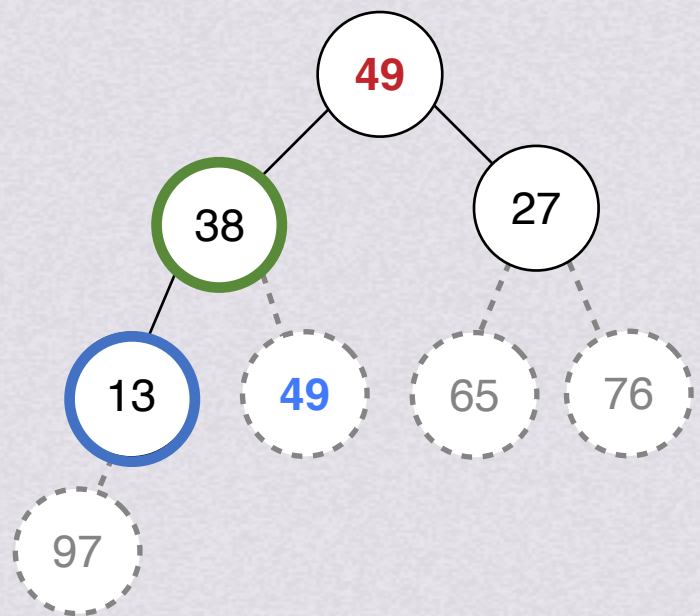
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

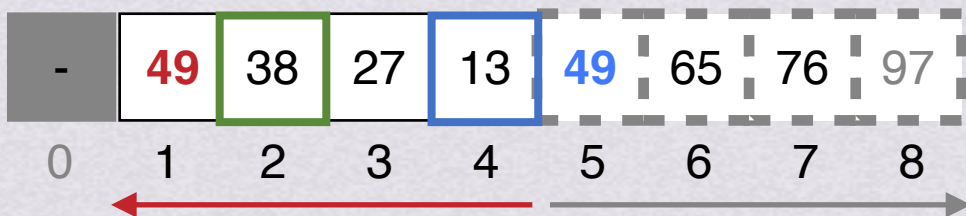
2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



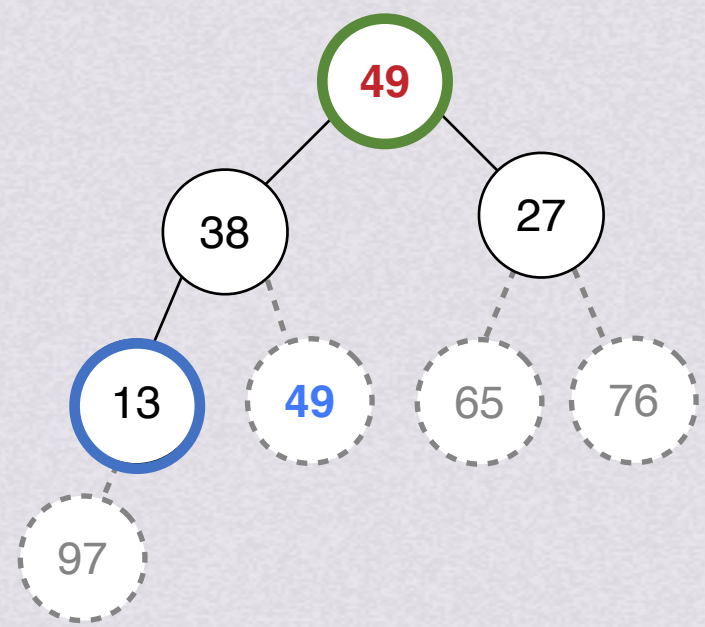
待排序元素 已有序元素

*0位置用于暂存根节点以完成交换操作，此处不更新

选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换
2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：
1.找到堆顶元素
2.找到堆底元素
3.交换
4.把剩余的i-1个元素调整成堆

调整完毕，继续交换堆顶堆底元素

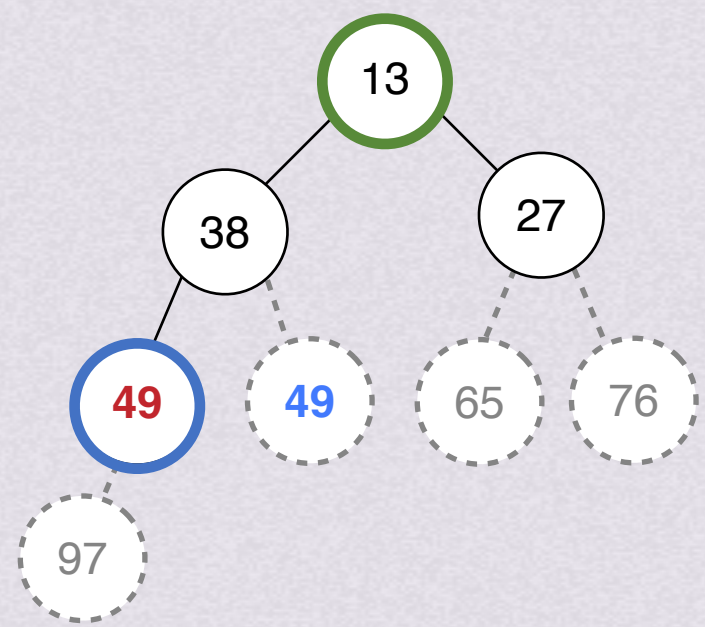


*0位置用于暂存根节点以完成交换操作，此处不更新

选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换
2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素
2.找到堆底元素
3.交换
4.把剩余的i-1个元素调整成堆

调整完毕，继续交换堆顶堆底元素



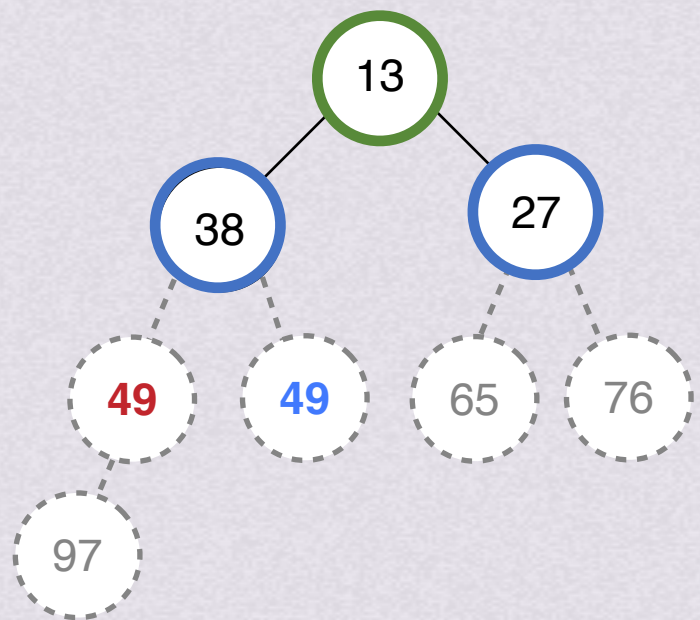
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



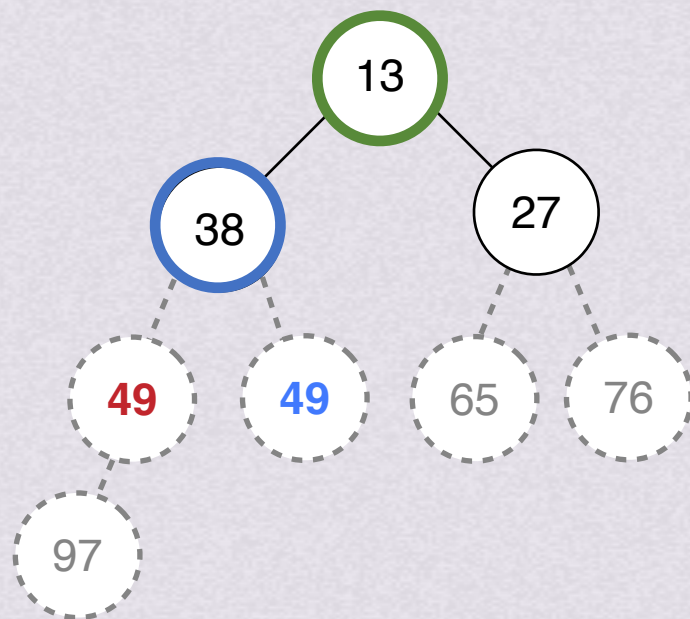
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



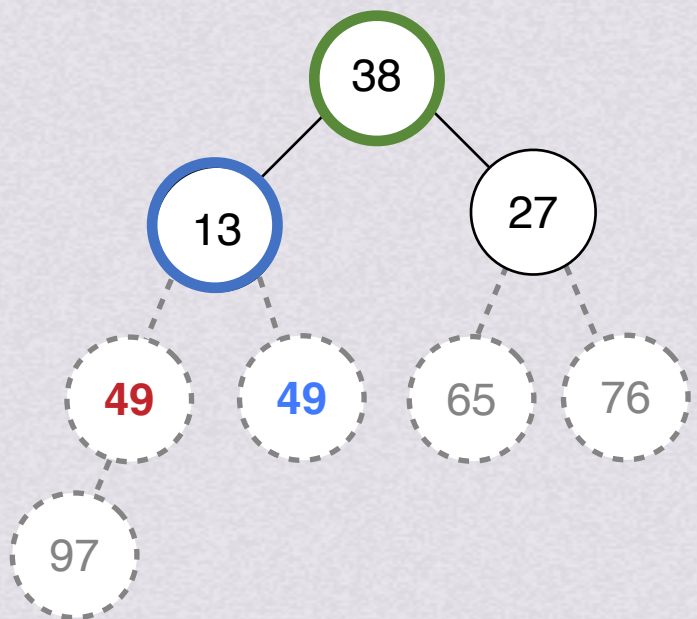
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换
2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素
2.找到堆底元素
3.交换

4.把剩余的i-1个元素调整成堆

调整完毕，继续交换堆顶堆底元素

无序序列

数组下标



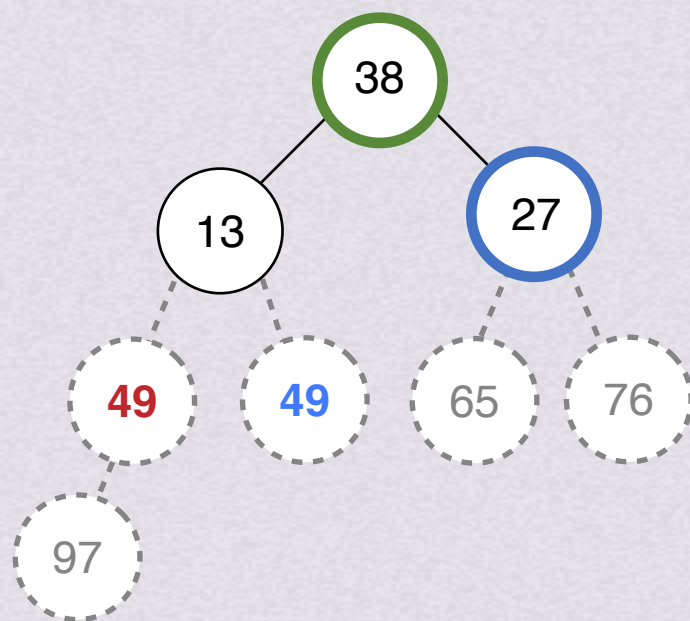
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：

1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的*i*-1个元素调整成堆

调整完毕，继续交换堆顶堆底元素

无序序列

数组下标



待排序元素 已有序元素

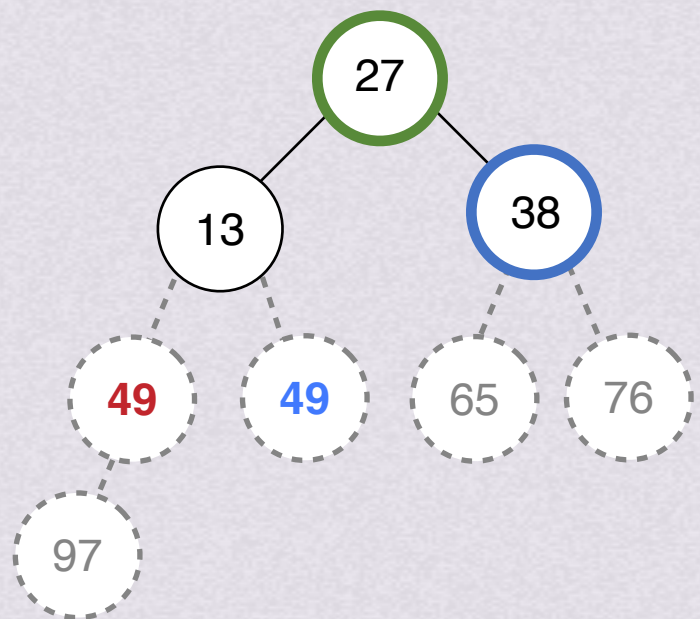
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标

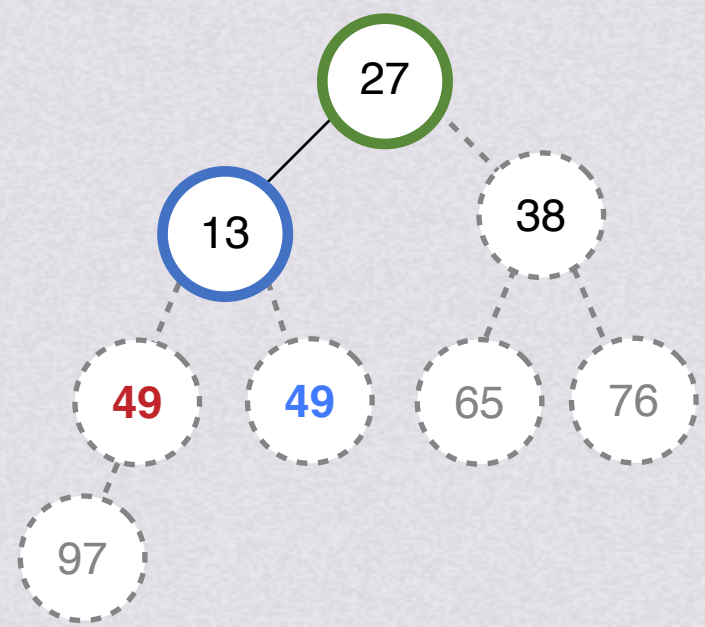


*0位置用于暂存根节点以完成交换操作，此处不更新

选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换
2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素
2.找到堆底元素
3.交换
4.把剩余的i-1个元素调整成堆

调整完毕，继续交换堆顶堆底元素



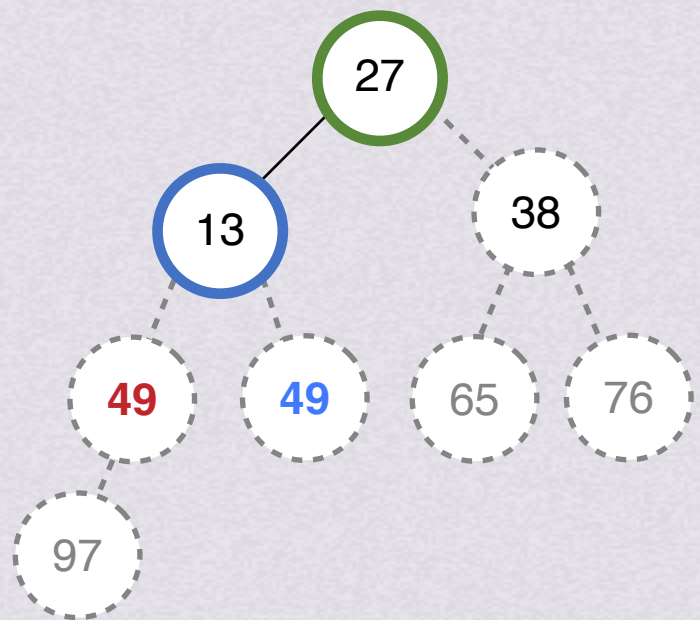
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：

1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



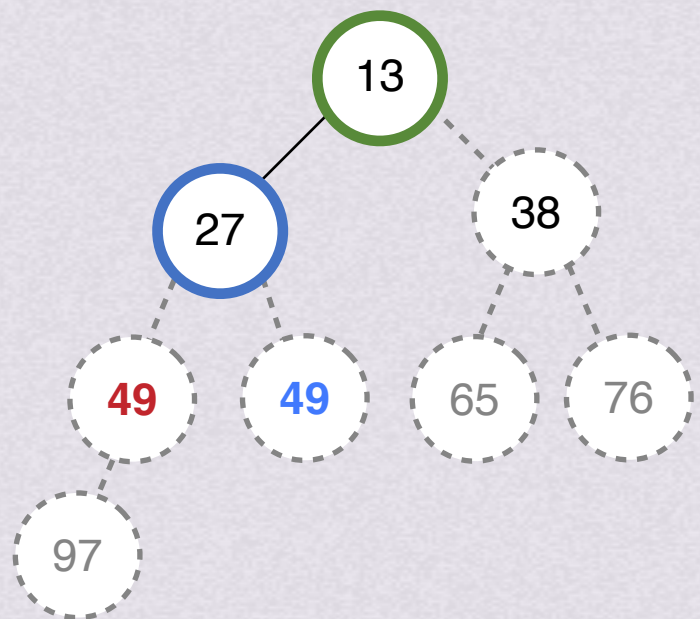
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



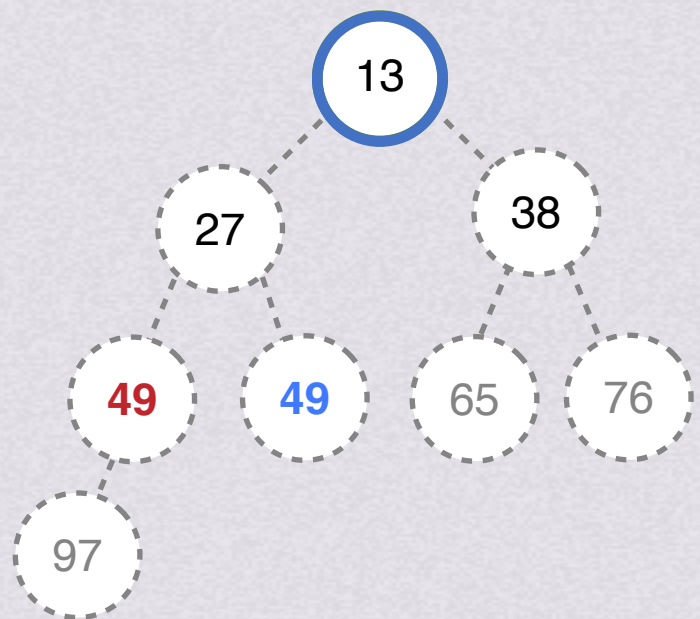
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

2.找到堆底元素

3.交换

4.把剩余的i-1个元素调整成堆

无序序列

数组下标



待排序元素 已有序元素

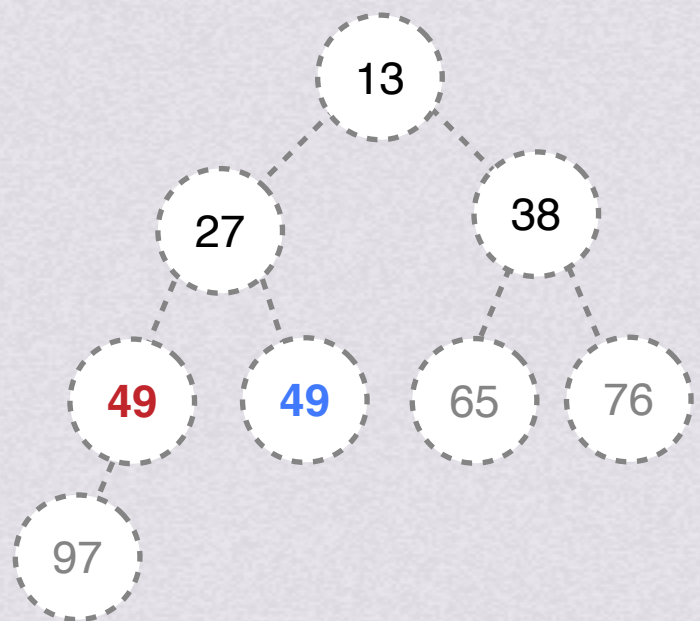
*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序堆排序

分为两个步骤：1.建堆

2.交换堆顶和堆底元素



大根堆建堆步骤：1.从最后一个分支节点所在子树开始
根节点与较大孩子互换

2.逐个从后往前让所有子树满足大根堆定义

交换元素步骤：1.找到堆顶元素

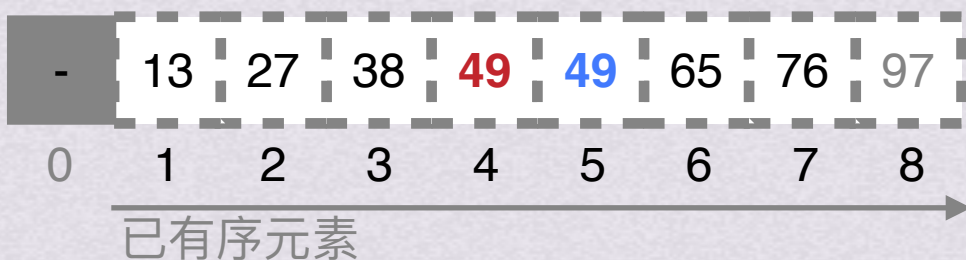
2.找到堆底元素

3.交换

4.把剩余的*i*-1个元素调整成堆

无序序列

数组下标



*0位置用于暂存根节点以完成交换操作，此处不更新



选择排序

堆排序

建堆代码

```
void CreatHeap(SqList &L)
{
    // 把无序序列 L[1..n] 建成大根堆
    n=L.length;
    for(i=n/2; i>0; --i) //反复调用HeapAdjust
        HeapAdjust(L, i, n);
}
```

从下标s开始对之后元素进行堆排序代码（堆调整代码）

```
void HeapAdjust(SqList &L, int s, int m)
{
    //假设L[s+1..m]已经是堆，将L[s..m]调整为以L[s]为根的大根堆
    rc=L[s];
    for(j=2*s; j<=m; j*=2) //沿key较大的孩子节点向下筛选
    {
        if(j<m && L[j]<L[j+1]) ++j; //j为key较大的记录的下标
        if(rc>=L[j]) break; //rc应插入在位置s上
        L[s]=L[j];
        s=j;
    } //for
    L[s]=rc; //插入
}
```

对顺序表L进行堆排序代码

```
void HeapSort(SqList &L)
{
    // 对顺序表 L 进行堆排序
    CreatHeap(L); //把无序序列L[1..L.length]建成大根堆
    for(i=L.length; i>1; --i)
    {
        x=L[1]; // 将堆顶记录和当前未经排序子序列 L[1..i] 中最后一个记录互换
        L[1]=L[i];
        L[i]=x;
        HeapAdjust(L, 1, i-1); // 将 L[1..i-1] 重新调整为大根堆
    } //for
}
```




选择排序堆排序

建堆代码

```
void CreatHeap(SqList &L)
{
    // 把无序序列 L[1..n] 建成大根堆
    n=L.length;
    for(i=n/2; i>0; --i) //反复调用HeapAdjust
        HeapAdjust(L, i, n);
}
```

1.初始化n为表长

2.从最后一棵树开始，不断向前遍历，并且调用堆调整的代码

从下标s开始对之后元素进行堆排序代码（堆调整代码）

```
void HeapAdjust(SqList &L, int s, int m)
{
    //假设L[s+1..m]已经是堆，将L[s..m]调整为以L[s]为根的大根堆
    rc=L[s];
    for(j=2*s; j<=m; j*=2) //沿key较大的孩子节点向下筛选
    {
        if(j<m && L[j]<L[j+1]) ++j; //j为key较大的记录的下标
        if(rc>=L[j]) break; //rc应插入在位置s上
        L[s]=L[j];
        s=j;
    } //for
    L[s]=rc; //插入
}
```

1.暂存根节点

2.不断将更大的孩子节点与根交换，交换完成后，顺着被交换的孩子继续向下调整
(通过下标*2的操作定位到左孩子)



选择排序堆排序

对顺序表L进行堆排序代码

1.对无序序列先建堆

2.建堆完成后，
不断将堆顶记录与堆底记录进行互换，
然后重新调整为堆

```
void HeapSort(SqList &L)
{
    // 对顺序表 L 进行堆排序
    CreatHeap(L); // 把无序序列L[1..L.length]建成大根堆
    for(i=L.length; i>1; --i)
    {
        x=L[1]; // 将堆顶记录和当前未经排序子序列 L[1..i] 中最后一个记录互换
        L[1]=L[i];
        L[i]=x;
        HeapAdjust(L, 1, i-1); // 将 L[1..i-1] 重新调整为大根堆
    } // for
}
```




选择排序堆排序

建堆代码

```
void CreatHeap(SqList &L)
{// 把无序序列 L[1..n] 建成大根堆
    n=L.length;
    for(i=n/2; i>0; --i) //反复调用HeapAdjust
        HeapAdjust(L, i, n);
}
```

从下标s开始对之后元素进行堆排序代码（堆调整代码）

```
void HeapAdjust(SqList &L, int s, int m)
{//假设L[s+1..m]已经是堆，将L[s..m]调整为以L[s]为根的大根堆
    rc=L[s];
    for(j=2*s; j<=m; j*=2) //沿key较大的孩子节点向下筛选
    {
        if(j<m && L[j]<L[j+1]) ++j; //j为key较大的记录的下标
        if(rc>=L[j]) break; //rc应插入在位置s上
        L[s]=L[j];
        s=j;
    } //for
    L[s]=rc; //插入
}
```

空间复杂度 $O(1)$

时间复杂度：

建堆时间 $O(n)$

每次向下调整 $O(h)$

最好，最坏，平均情况下，均为 $O(n\log_2 n)$

不稳定！

对顺序表L进行堆排序代码

```
void HeapSort(SqList &L)
{// 对顺序表 L 进行堆排序
    CreatHeap(L); //把无序序列L[1..L.length]建成大根堆
    for(i=L.length; i>1; --i)
    {
        x=L[1]; // 将堆顶记录和当前未经排序子序列 L[1..i] 中最后一个记录互换
        L[1]=L[i];
        L[i]=x;
        HeapAdjust(L, 1, i-1); // 将 L[1..i-1] 重新调整为大根堆
    } //for
}
```